



Chương 8: Phân loại Ảnh với Mạng Tích chập (Image Classification with ConvNets)

Học Sâu (Deep Learning)

Đại học Đông Á | August 14, 2026

Nội dung chương



Nền tảng của Mạng Tích chập (CNN)



Thực hành: Bài toán Phân loại Chó & Mèo



Chống Overfitting bằng Data Augmentation



Chuyển giao Học tập (Transfer Learning)



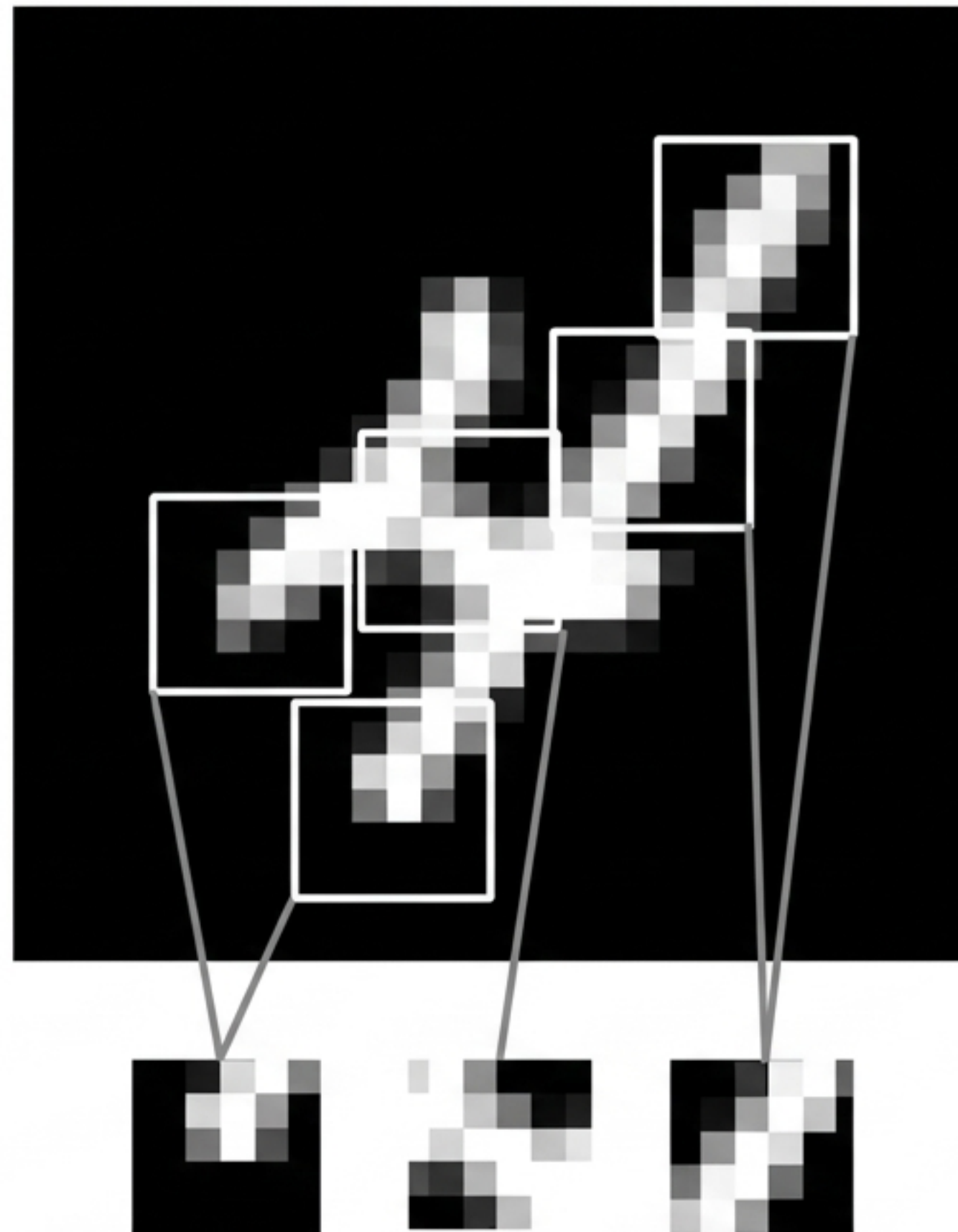
Tinh chỉnh (Fine-Tuning)



Tổng kết

Tại sao lại là Tích chập (Convolution)?

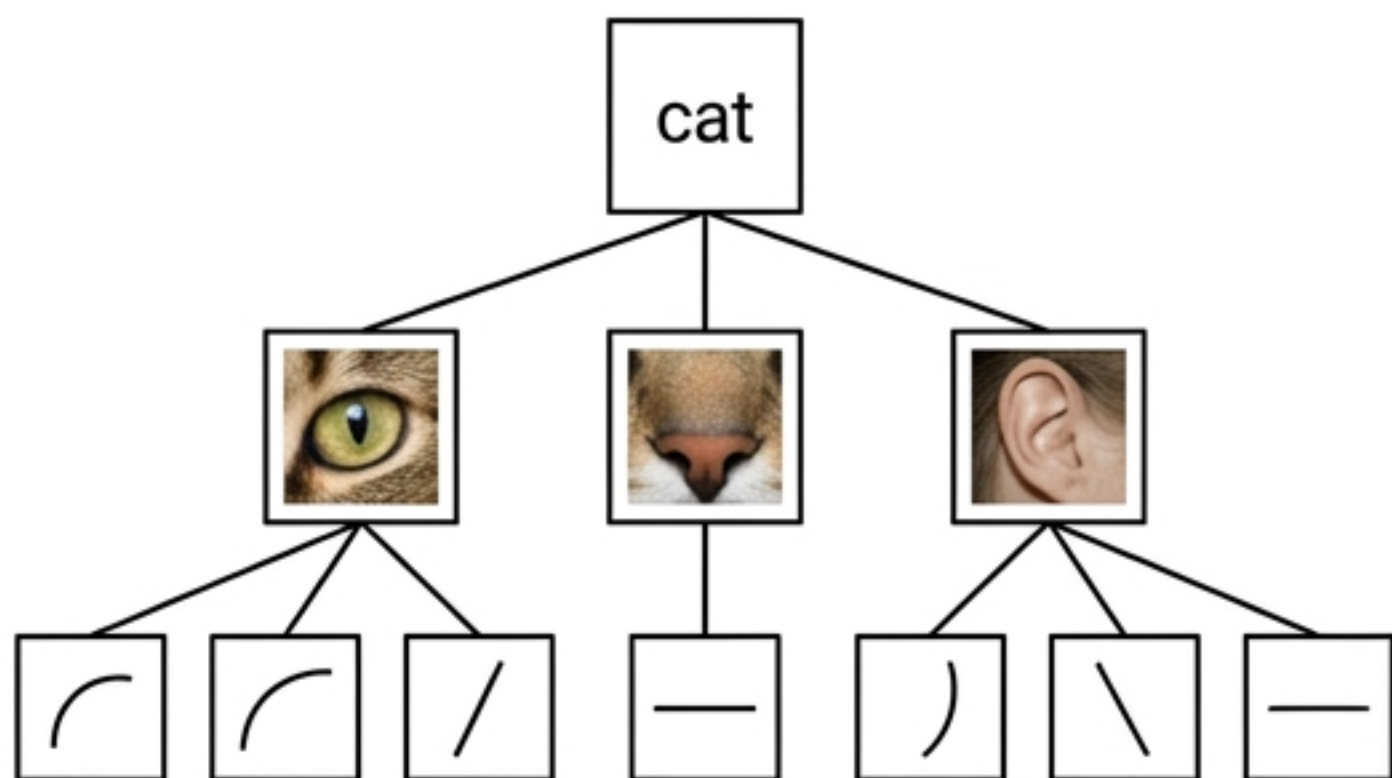
- Mạng Dense truyền thống: Tìm kiếm các mẫu toàn cục (Global Patterns) trên toàn bộ bức ảnh.
- Mạng Tích chập (ConvNets): Tìm kiếm các mẫu cục bộ (Local Patterns) như cạnh, góc, kết cấu.
- Tính bất biến dịch chuyển (Translation Invariance): Nếu CNN học được hình dáng con con mắt ở góc trái ảnh, nó vẫn có thể nhận ra con mắt đó ở góc phải ảnh. Dense Layer thì không!



Hình 8.1: Local Patterns vs Global Patterns

Hệ thống Phân cấp Thị giác (Visual Hierarchy)

- CNN học theo một hệ thống phân cấp tự nhiên:
 - **Lớp đầu tiên:** Học các đặc trưng đơn giản (Cạnh, Góc).
 - **Lớp giữa:** Ghép các cạnh thành bộ phận (Mắt, Mũi, Lưỡi).
 - **Lớp cuối:** Tổng hợp thành vật thể hoàn chỉnh (Con mèo, Con chó).
- Điều này **giống hệt** cách vỏ não thị giác (Visual Cortex) của con người hoạt động.



Hình 8.2: Sự kết hợp các đặc trưng bậc thấp thành bậc cao



Mạng CNN cơ bản: Mã nguồn

Cấu trúc một mạng phân loại ảnh đơn giản bao gồm việc xen kẽ các lớp **Conv2D** và **MaxPooling2D**.

```
import keras
from keras import layers

inputs = keras.Input(shape =(28, 28, 1))
x = layers.Conv2D(filters =64, kernel_size =3, activation='relu')(inputs)
x = layers.MaxPooling2D(pool_size =2)(x)
x = layers.Conv2D(filters =128, kernel_size =3, activation='relu')(x)
x = layers.MaxPooling2D(pool_size =2)(x)
x = layers.Conv2D(filters =256, kernel_size =3, activation='relu')(x)
x = layers.GlobalAveragePooling2D ()(x)
outputs = layers.Dense (10, activation='softmax')(x)

model = keras.Model(inputs=inputs , outputs=outputs)
```

Kiểm tra quá trình nén không gian (Model Summary)

Kích thước không gian (chiều rộng × chiều cao) giảm dần, trong khi chiều sâu (số kênh) tăng dần.

```
>>> model.summary ()
Model: 'functional'
...
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None , 28, 28, 1)	0
conv2d (Conv2D)	(None , 26, 26, 64)	640
max_pooling2d (MaxPooling2D)	(None , 13, 13, 64)	0
conv2d_1 (Conv2D)	(None , 11, 11, 128)	73856
max_pooling2d_1 (MaxPooling2D)	(None , 5, 5, 128)	0
conv2d_2 (Conv2D)	(None , 3, 3, 256)	295168
global_average_pooling2d	(None , 256)	0
dense (Dense)	(None , 10)	2570

```
...
```

Toán học của Phép Tích chập (Dot Product)

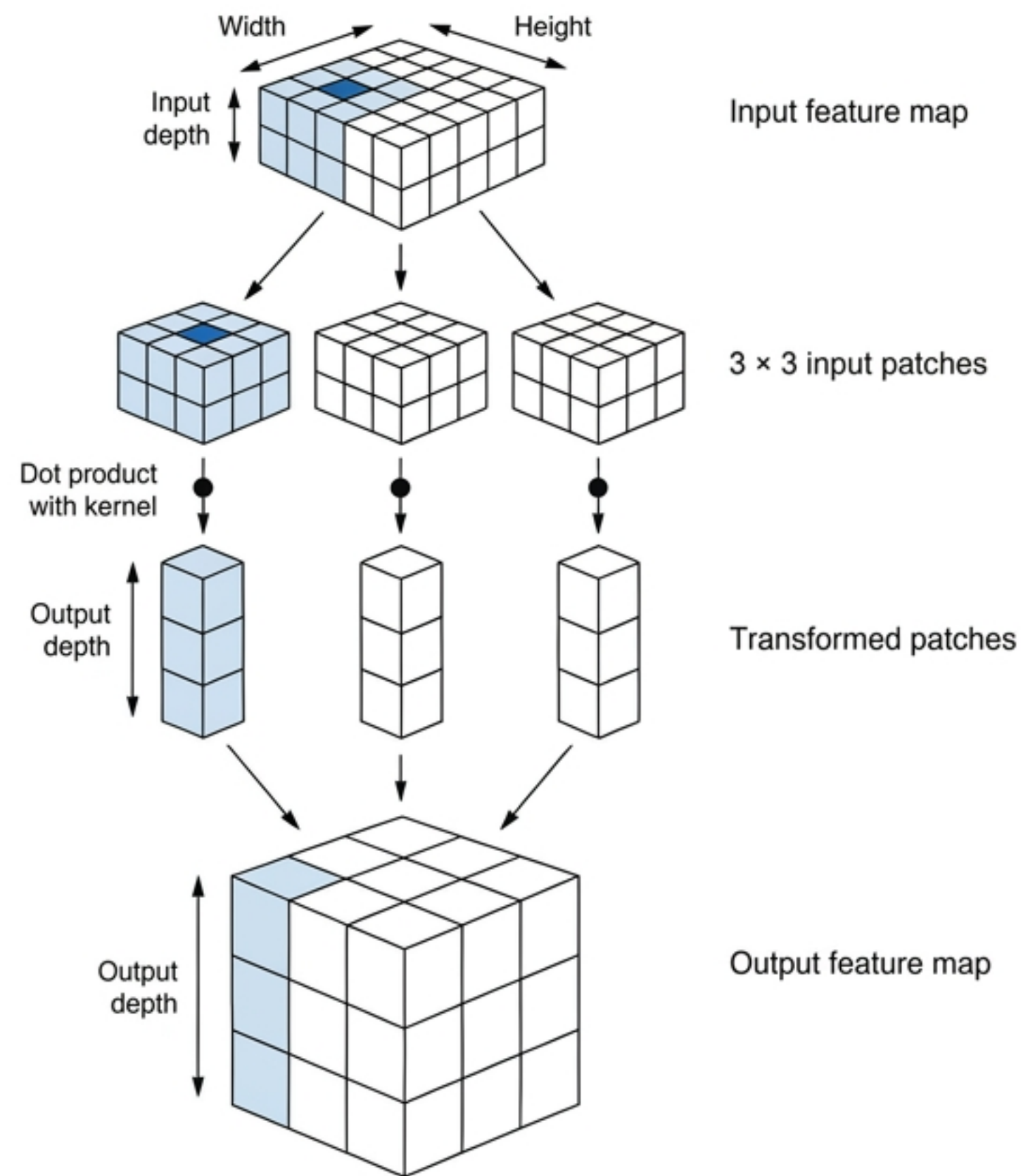
Toán tử Tích chập 2D được định nghĩa:

$$\begin{aligned} S(i, j) &= (I * K)(i, j) \\ &= \sum_m \sum_n I(i-m, j-n) K(m, n) \end{aligned}$$

I : Ma trận ảnh đầu vào.

K : Bộ lọc (Kernel / Filter).

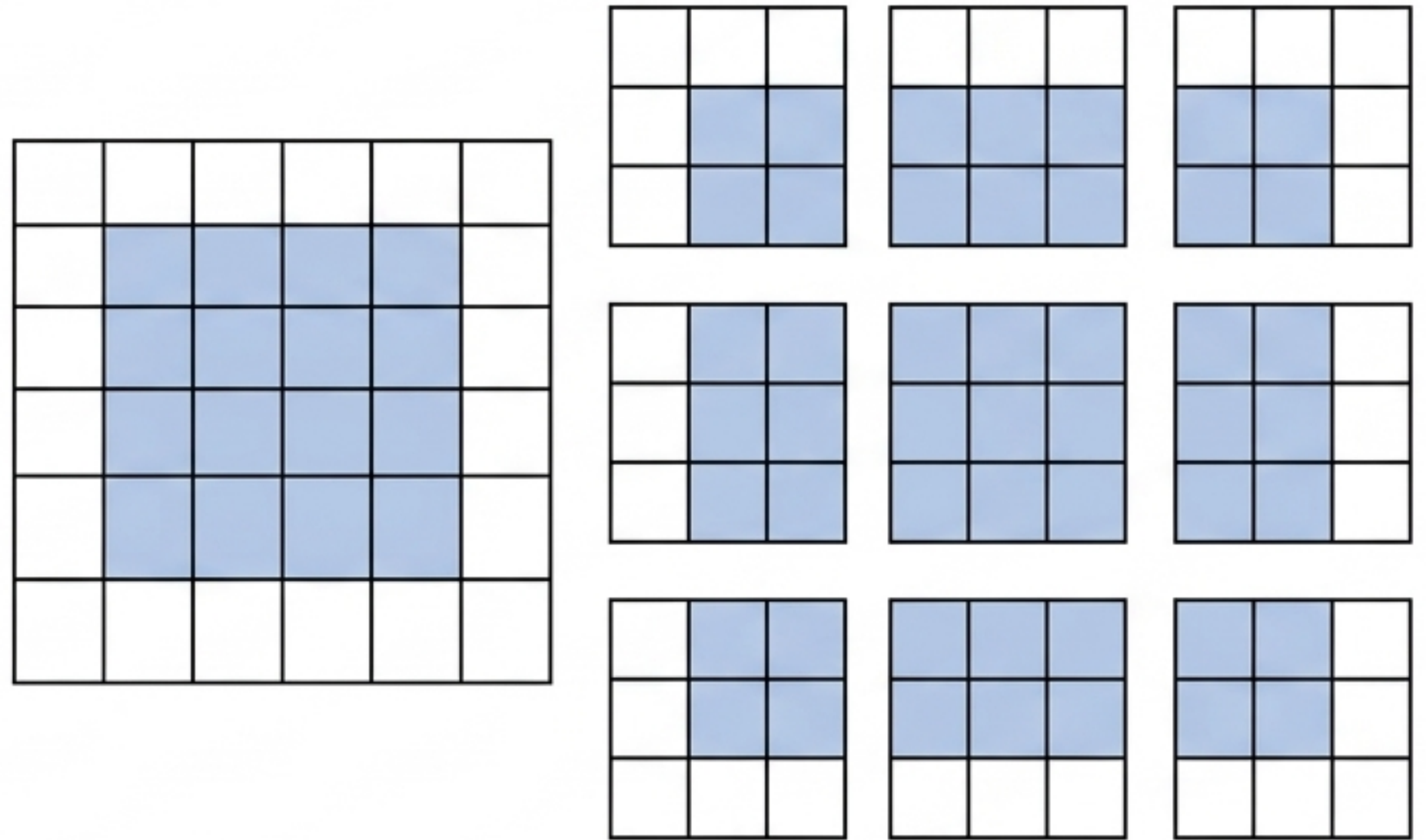
Bản chất là trượt bộ lọc K trên toàn bộ ảnh và tính Tích vô hướng (Dot Product).



Hình 8.3: Phép trượt nhân chập cơ bản

Trích xuất Phân mảnh (Patches)

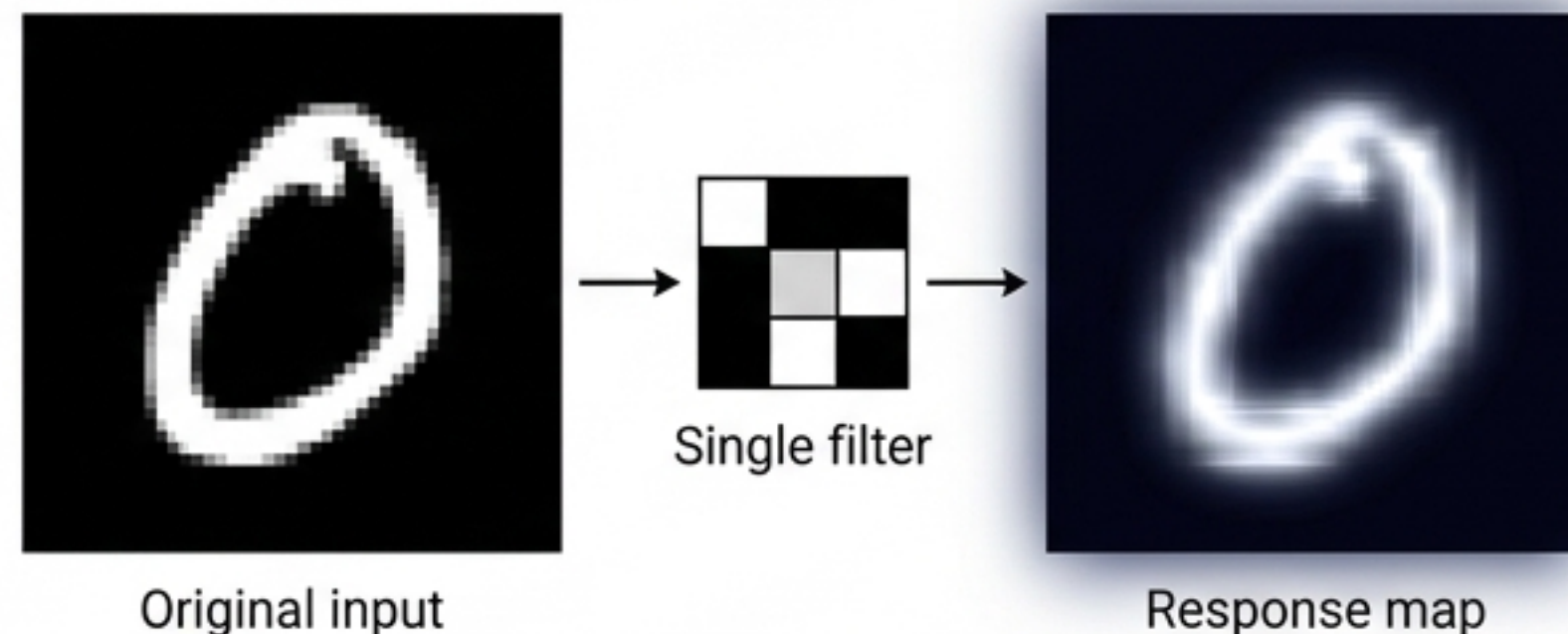
- Nếu có ảnh đầu vào kích thước 5×5 , và sử dụng Bộ lọc kích thước 3×3 .
- Bộ lọc sẽ cắt ra các phân mảnh (patches) có cùng kích thước 3×3 .
- Khớp với mỗi phân mảnh, phép Tích vô hướng sẽ sinh ra duy nhất một con số (Scalar).



Hình 8.4: Trích xuất các patches 3×3

Bản đồ Phản hồi (Response Map)

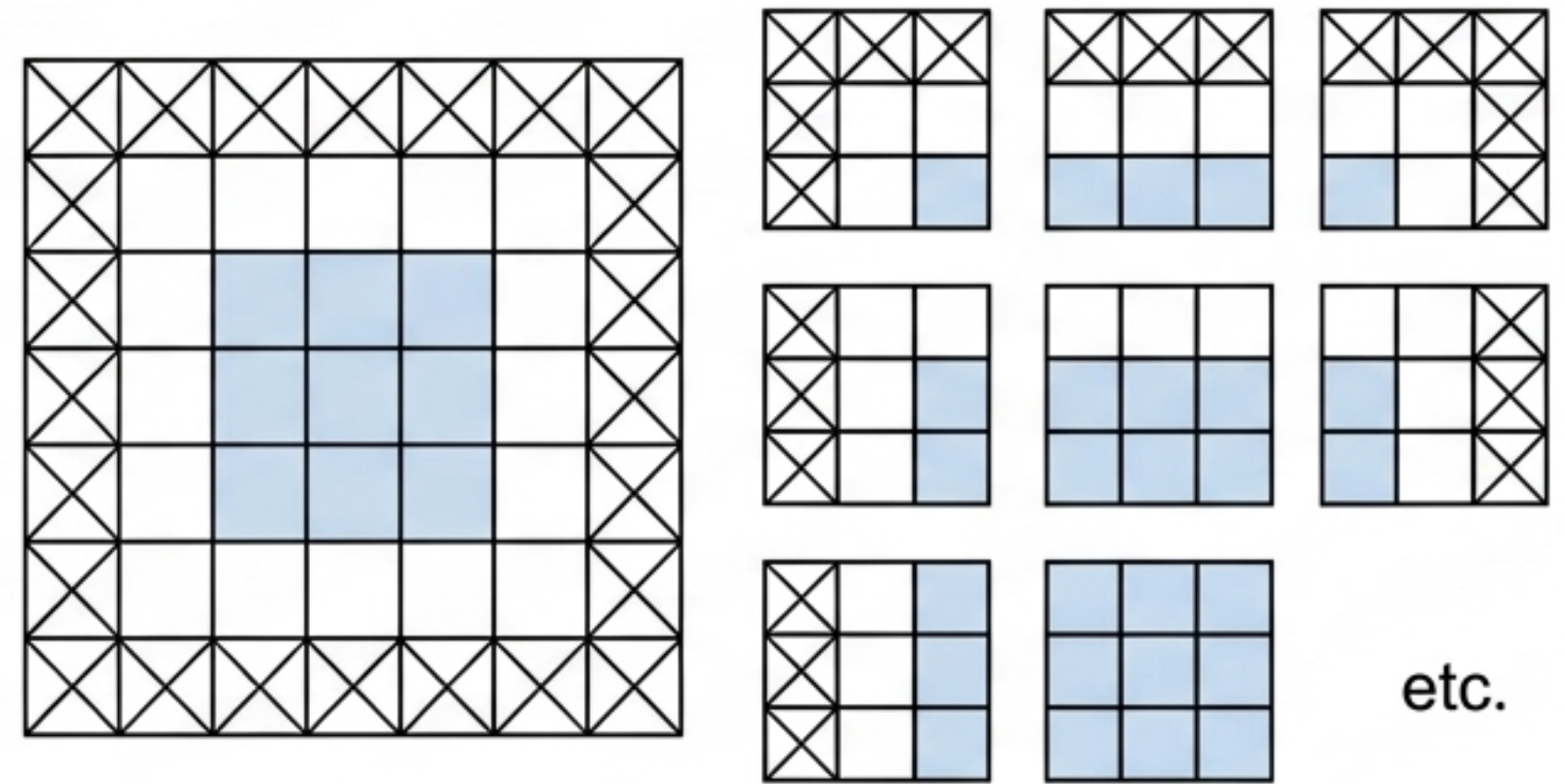
- Mỗi Bộ lọc (Filter) đại diện cho một "Khái niệm" (Vd: Khái niệm "có mắt mèo trong hình không?").
- Kết quả của quá trình tích chập tạo ra một Bản đồ phản hồi (Response Map).
- Điểm nào sáng trên bản đồ nghĩa là Bộ lọc đã "nhìn thấy" mẫu mà nó đang tìm kiếm tại vị trí đó.



Hình 8.5: Một bản đồ phản hồi đặc trưng

Kỹ thuật Lấp đầy (Padding)

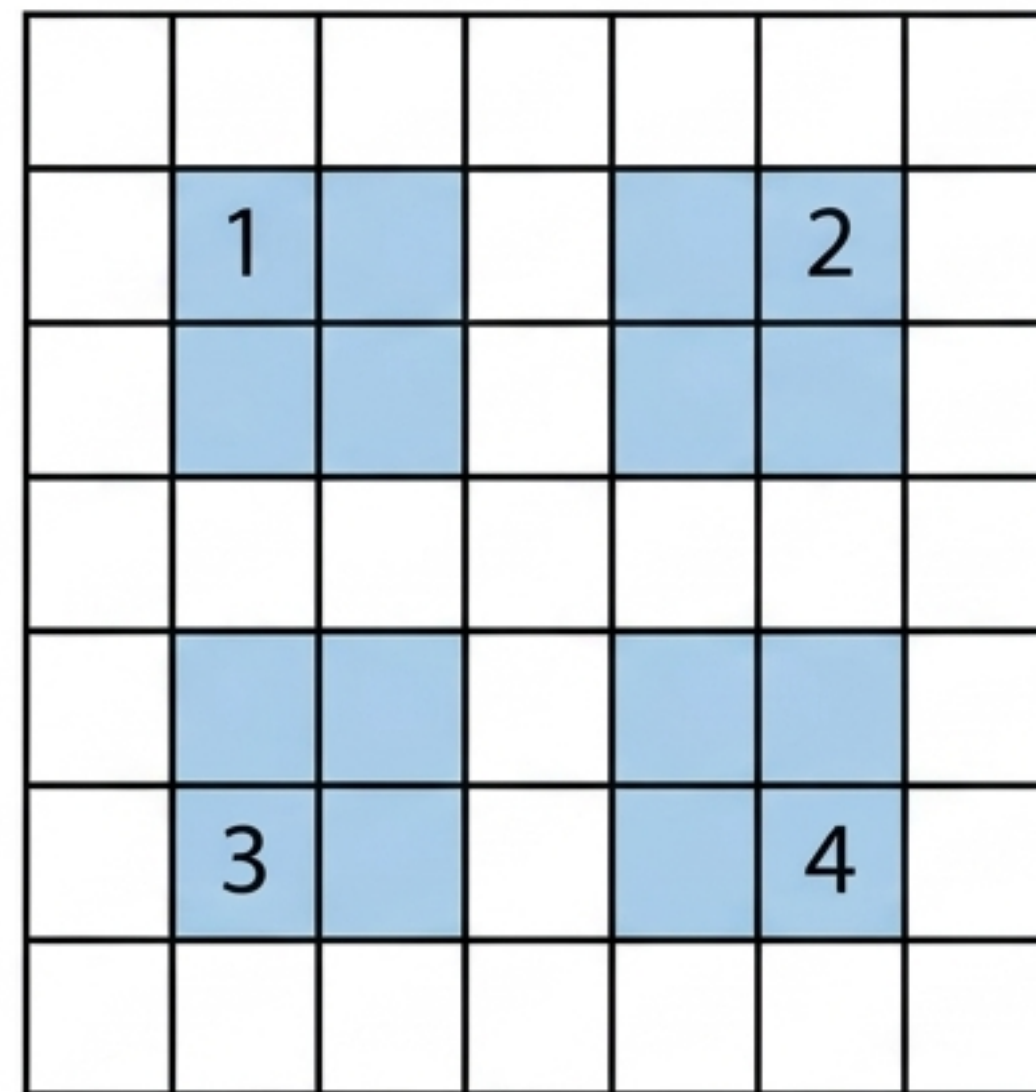
- Vấn đề: Kích thước ảnh luôn bị co hẹp lại sau khi Tích chập (Từ 5×5 xuống 3×3). Lớp biên của ảnh gốc bị bỏ qua rất nhiều.
- Giải pháp (Padding): Thêm các viền số 0 (Zero-padding) bao quanh ảnh để bảo toàn kích thước sau khi quét.
- Tham số Keras: padding="same" (Bảo toàn) hoặc padding="valid" (Không lấp đầy).



Hình 8.6: Minh họa Zero Padding

Bước nhảy (Strides)

- Stride: Khoảng cách (số pixel) mà Bộ lọc trượt đi sau mỗi nhịp. Mặc định là $\text{strides}=1$.
- Nếu $\text{strides}=2$, bộ lọc sẽ nhảy cóc qua 1 pixel. Kích thước bản đồ đầu ra sẽ giảm đi chính xác một nửa (Downsampling).
- Lưu ý: Trong thực tế ta thường dùng MaxPooling để giảm kích thước thay vì Strides.



Hình 8.7: Bộ lọc trượt với Strides bằng 2

Tập Dữ liệu Kaggle Dogs vs Cats



- Bài toán thi đấu nổi tiếng năm 2013 trên Kaggle.
- Cung cấp 25,000 ảnh chó và mèo nhiều kích cỡ, màu sắc, bối cảnh.
- Mục tiêu thực hành: Ta sẽ trích xuất ra một tập con siêu nhỏ (Chỉ 2,000 ảnh cho Training) để giả lập kịch bản thiếu dữ liệu rất biến trong doanh nghiệp.



Hình 8.8: Mẫu ảnh từ bộ dữ liệu

Tiền xử lý ảnh với Keras

Với những dữ liệu lớn, việc đọc tất cả vào RAM là bất khả thi. `image_dataset_from_directory` tạo một Data Generator lấy ảnh trực tiếp từ ổ cứng theo generator lấy ảnh trực tiếp từ ổ cứng theo từng lô.

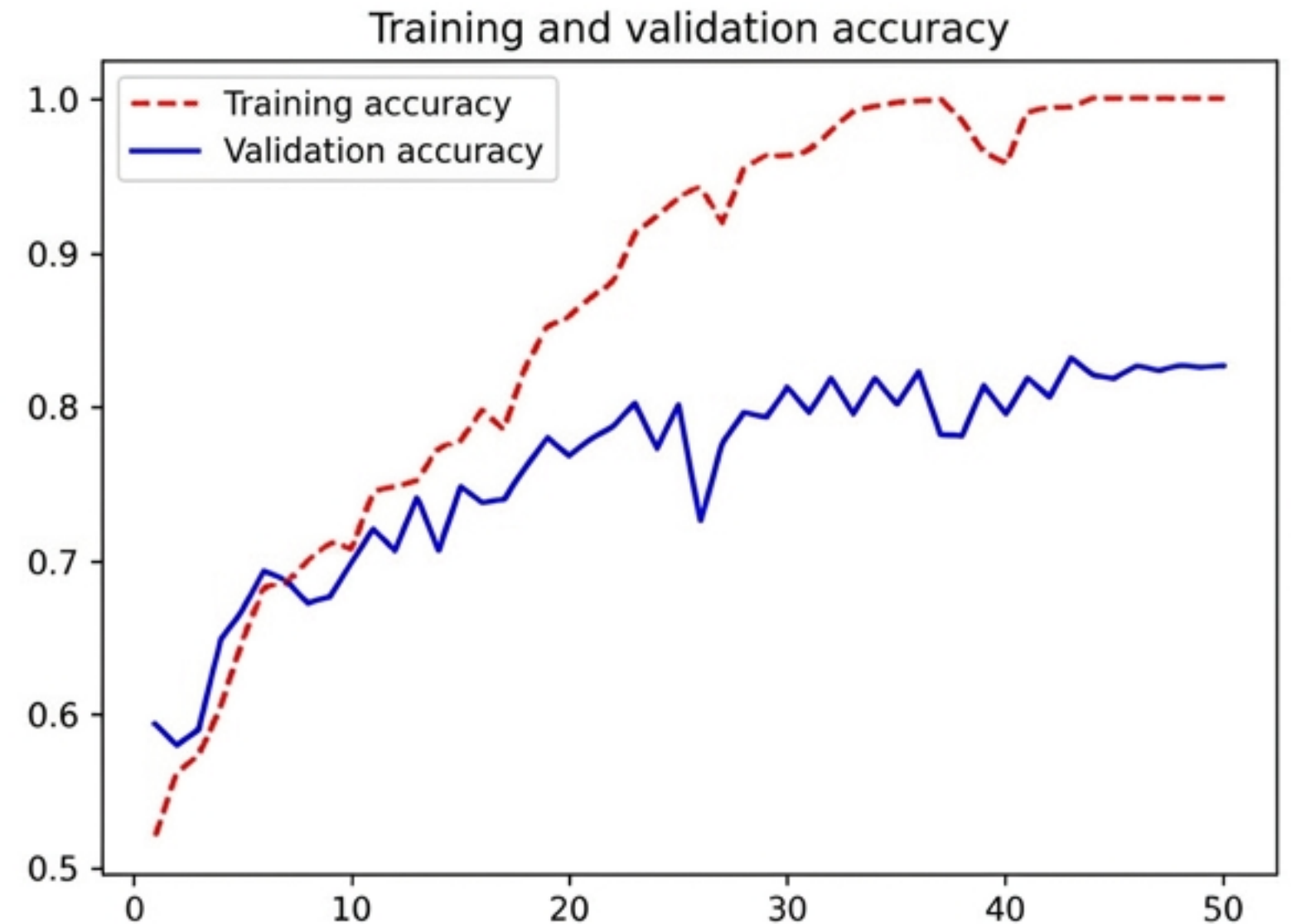
```
from keras.utils import image_dataset_from_directory

train_dataset = image_dataset_from_directory(
    'dogs_vs_cats_small/train',
    image_size=(180, 180),
    batch_size=32
)

validation_dataset = image_dataset_from_directory(
    'dogs_vs_cats_small/validation',
    image_size=(180, 180),
    batch_size=32
)
```

Huấn luyện CNN cơ bản (Từ con số 0)

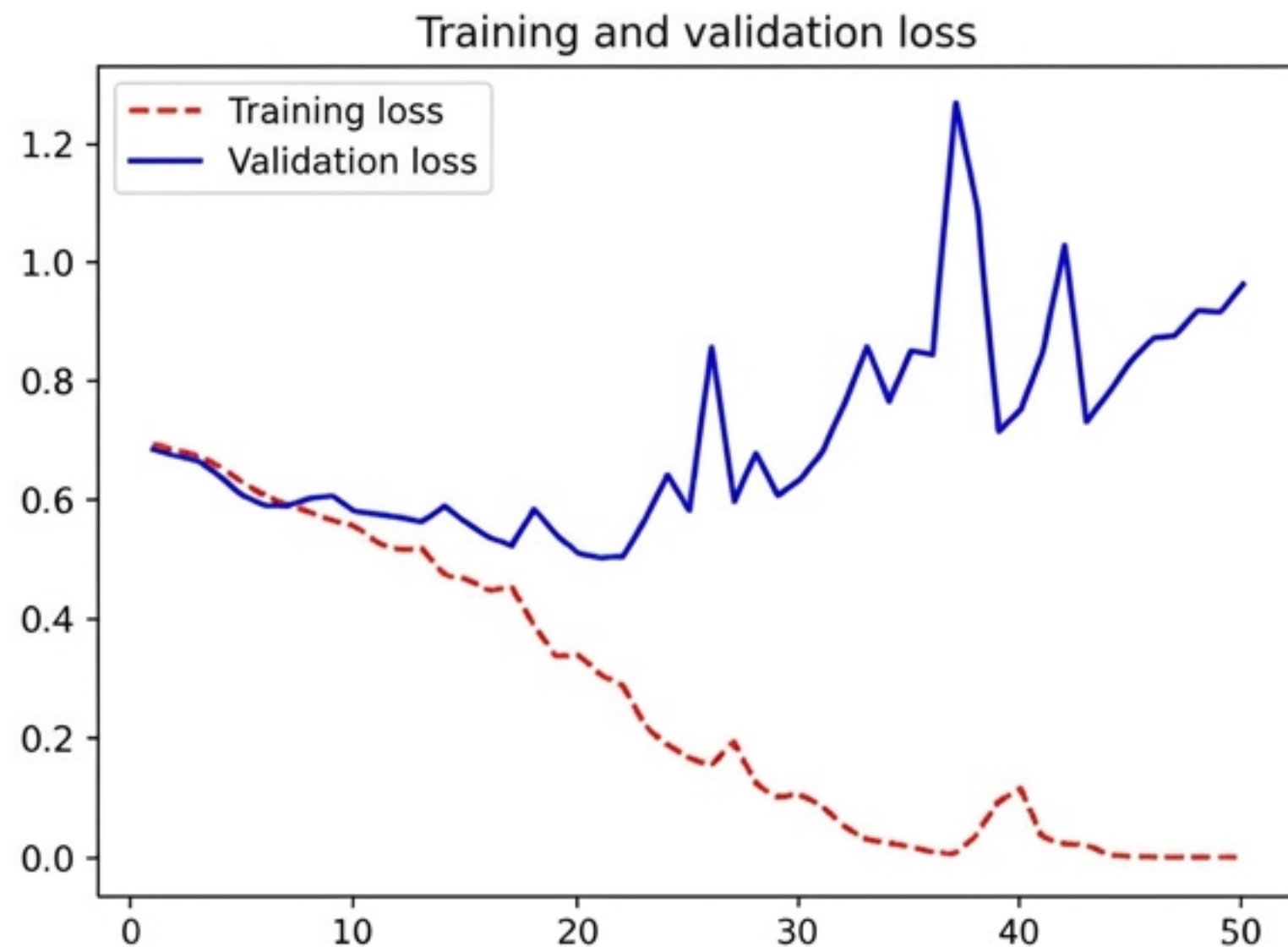
- Thiết kế mạng với 4 khối Conv2D + MaxPooling2D.
- Kết quả: Validation Accuracy đạt khoảng 70-75% rồi chững lại.
- Khoảng cách giữa đường Train (Xanh) và Validation (Đứt nét) nới rộng chóng → Dấu hiệu Overfitting điển hình.



Hình 8.9: Accuracy Mô hình cơ bản

Phân tích hàm Loss cơ bản

- Hàm mất mát Training liên tục giảm tiến về 0. Mô hình đang "thuộc lòng" (Memorizing) dữ liệu học.
- Hàm mất mát Validation không thể giảm xuống dưới ngưỡng 0.8 và bắt đầu bật tăng ngược trở lại sau Epoch 10.
- Kết luận: Do dữ liệu huấn luyện quá ít (2000 ảnh), mạng CNN không thể khái quát hóa. Ta cần kỹ thuật mạnh hơn.



Hình 8.10: Loss Mô hình cơ bản